

Updates

Lhamo Dorje

I. OPTIMIZATION MODIFICATIONS ACROSS THE THREE DEEP2S DI-ATTACK VARIANTS

For the Deep2S attack, I updated how the complex gains A_p are optimized in order to obtain a more stable attack under the same physical amplitude constraint $|A_p| \leq A_{\max}$.

Old version (baseline). In the original implementation, I optimized the real and imaginary parts of the gains directly:

$$A_{\text{re}} \leftarrow A_{\text{re}} - \eta_{\text{re}} \nabla_{A_{\text{re}}} \mathcal{L}, \quad A_{\text{im}} \leftarrow A_{\text{im}} - \eta_{\text{im}} \nabla_{A_{\text{im}}} \mathcal{L},$$

using relatively large learning rates. After each gradient step, I enforced the amplitude constraint via a hard projection: for each aperture p , I computed $|A_p|$ and, whenever $|A_p| > A_{\max}$, rescaled A_p back to $|A_p| = A_{\max}$. This procedure caused many entries to sit exactly at the bound $|A_p| = A_{\max}$ and led to oscillatory loss behavior due to the combination of large step sizes and abrupt clipping.

New version (current) In the revised implementation, I optimize unconstrained latent variables $Z_{\text{re}}, Z_{\text{im}}$ and map them smoothly to A :

$$A_{\text{re}} = \frac{A_{\max}}{\sqrt{2}} \tanh(Z_{\text{re}}), \quad A_{\text{im}} = \frac{A_{\max}}{\sqrt{2}} \tanh(Z_{\text{im}}),$$

so that the complex gains are

$$A = A_{\text{re}} + jA_{\text{im}}.$$

This guarantees $|A_p| \leq A_{\max}$ by construction (no explicit clipping) and preserves useful gradients near the boundary. Instead of hand-rolled gradient descent, I now use the Adam optimizer on $Z_{\text{re}}, Z_{\text{im}}$, which provides adaptive step sizes and smoother convergence. I retain an ℓ_2 penalty on $|A|$ (through the parameter `lambda_L2`) so that the optimizer does not simply push all gains to their maximum amplitude, but rather uses the available power budget in a more controlled and structured way.

In this revised formulation, the attack loss $\mathcal{L}$ is still computed in the image domain based on A (through the attacked measurements, Deep2S forward model, and MSE to the target volume), but during backpropagation we differentiate with respect to $Z_{\text{re}}, Z_{\text{im}}$. By the chain rule,

$$\frac{\partial \mathcal{L}}{\partial Z_{\text{re}}} = \frac{\partial \mathcal{L}}{\partial A_{\text{re}}} \cdot \frac{\partial A_{\text{re}}}{\partial Z_{\text{re}}} = \frac{\partial \mathcal{L}}{\partial A_{\text{re}}} \frac{A_{\max}}{\sqrt{2}} (1 - \tanh^2(Z_{\text{re}})),$$

$$\frac{\partial \mathcal{L}}{\partial Z_{\text{im}}} = \frac{\partial \mathcal{L}}{\partial A_{\text{im}}} \cdot \frac{\partial A_{\text{im}}}{\partial Z_{\text{im}}} = \frac{\partial \mathcal{L}}{\partial A_{\text{im}}} \frac{A_{\max}}{\sqrt{2}} (1 - \tanh^2(Z_{\text{im}})).$$

The optimizer (Adam in our implementation) then updates $Z_{\text{re}}, Z_{\text{im}}$:

$$Z_{\text{re}} \leftarrow Z_{\text{re}} - \alpha \text{Adam}(\nabla_{Z_{\text{re}}} \mathcal{L})$$

$$Z_{\text{im}} \leftarrow Z_{\text{im}} - \alpha \text{Adam}(\nabla_{Z_{\text{im}}} \mathcal{L})$$

Algorithm 1 Compressed Sensing Algorithm (CSA) in H, x notation

Require: Raw echo $\mathbf{y}_s$, measurement matrix $\mathbf{H}$

Ensure: 3-D imaging result cube $\hat{\mathbf{x}}$

1: Initialize parameters λ_0, p , and η , iterative threshold ε_0

2: $\hat{\mathbf{x}}^{(0)} = \mathbf{H}^H \mathbf{y}_s$, set $n = 0$

3: $r = \frac{\|\hat{\mathbf{x}}^{(n)} - \hat{\mathbf{x}}^{(n-1)}\|_2}{\|\hat{\mathbf{x}}^{(n)}\|_2}$ $\triangleright$ defined for $n \geq 1$

4: **while** $r \geq \varepsilon_0$ **do**

5: Diagonal matrix $\mathbf{\Lambda}^{(n)}$ with entries

$$\Lambda_{ii}^{(n)} = \frac{p}{2} \left(|\hat{x}_i^{(n-1)}|^2 + \eta \right)^{\frac{p}{2}-1}$$

6: Update image estimate

$$\hat{\mathbf{x}}^{(n)} = \left(\mathbf{H}^H \mathbf{H} + \lambda_0 \beta^{(n)} \mathbf{\Lambda}^{(n)} \right)^{-1} \mathbf{H}^H \mathbf{y}_s$$

7: Update noise parameter

$$\beta^{(n)} = \frac{\|\mathbf{y}_s - \mathbf{H} \hat{\mathbf{x}}^{(n)}\|_2^2}{N}$$

8: $n \leftarrow n + 1$

9: $r = \frac{\|\hat{\mathbf{x}}^{(n)} - \hat{\mathbf{x}}^{(n-1)}\|_2}{\|\hat{\mathbf{x}}^{(n)}\|_2}$

10: **end while**

11: $\hat{\mathbf{x}} \leftarrow \hat{\mathbf{x}}^{(n)}$

or in general

$$Z_{\text{re}}^{(n+1)} = Z_{\text{re}}^{(n)} - \eta \frac{A_{\max}}{\sqrt{2}} \frac{\partial \mathcal{L}}{\partial A_{\text{re}}} (1 - \tanh^2(Z_{\text{re}}^{(n)})),$$

$$Z_{\text{im}}^{(n+1)} = Z_{\text{im}}^{(n)} - \eta \frac{A_{\max}}{\sqrt{2}} \frac{\partial \mathcal{L}}{\partial A_{\text{im}}} (1 - \tanh^2(Z_{\text{im}}^{(n)})),$$

and after each update we recompute $A_{\text{re}}, A_{\text{im}}$ via the tanh mapping. This guarantees $|A_p| \leq A_{\max}$ by construction, while still allowing gradients to flow smoothly near the amplitude bound.

Overall, the change is from a “direct + hard-clipping” optimization of A with large steps to a smoothly constrained, Adam-based optimization in the latent space ($Z_{\text{re}}, Z_{\text{im}}$), under the same physical amplitude bound $|A_p| \leq 2$.

II. “LINEAR TIKHONOV INVERSE” SURROGATE IN DI-ATTACK ON CSA

In the CSA formulation, SAR measurements are modeled as

$$\mathbf{y}_s = \mathbf{H} \mathbf{x} + \mathbf{n}, \quad (1)$$

where $\mathbf{y}_s \in \mathbb{C}^{M_{\text{meas}}}$ are the echoes, $\mathbf{x} \in \mathbb{C}^{N_{\text{pix}}}$ is the unknown reflectivity, $\mathbf{H}$ is the propagation matrix, and $\mathbf{n}$ is noise.

Sparse prior and MAP objective CSA assumes a sparse scene,

$$p(\mathbf{x}) \propto \prod_{i=1}^{N_{\text{pix}}} \exp(-\lambda_0 |x_i|^p), \quad 0 < p \leq 1, \quad (2)$$

leading to the MAP problem

$$\hat{\mathbf{x}} = \arg \min_{\mathbf{x}} \|\mathbf{y}_s - \mathbf{H}\mathbf{x}\|_2^2 + \lambda_0 \beta \|\mathbf{x}\|_p^p, \quad (3)$$

where β is an inverse noise-variance estimate.

SBRIM iterations (original CSA) CSA solves (3) via SBRIM. At iteration n ,

$$\Lambda_{ii}^{(n)} = \frac{p}{2} (|x_i^{(n-1)}|^2 + \eta)^{\frac{p}{2}-1}, \quad (4)$$

$$x^{(n)} = \left(\mathbf{H}^H \mathbf{H} + \lambda_0 \beta^{(n)} \Lambda^{(n)} \right)^{-1} \mathbf{H}^H \mathbf{y}_s, \quad (5)$$

$$\beta^{(n)} = \frac{\|\mathbf{y}_s - \mathbf{H}x^{(n)}\|_2^2}{M_{\text{meas}}}. \quad (6)$$

Iterations stop when $r^{(n)} = \|x^{(n)} - x^{(n-1)}\|_2 / \|x^{(n)}\|_2 < \varepsilon_0$, and the final CSA reconstruction is $\hat{\mathbf{x}} = x^{(n_{\text{final}})}$. Each step solves a regularized least-squares system with an adaptively reweighted diagonal penalty. Large coefficients are penalized less and small ones more, promoting sparsity in $\hat{\mathbf{x}}$.

Original CSA vs. Our Attack Implementation

Victim mapping: the victim we attack is the original CSA/SBRIM mapping

$$\mathbf{y}_s \mapsto \hat{\mathbf{x}}_{\text{CSA}} = \text{SBRIM}(\mathbf{y}_s, \mathbf{H}, \lambda_0, p, \eta),$$

implemented exactly as in the equations above.

We inject a complex gain vector $A \in \mathbb{C}^{N_p}$ into a bank of time-domain attack waveforms $D \in \mathbb{C}^{N_{\text{samp}} \times N_p}$, added to the clean raw data $X_v \in \mathbb{C}^{N_{\text{samp}} \times N_p}$. After SAR preprocessing (FFT, range-bin selection, row flipping), this yields attacked CSA measurements

$$\mathbf{y}_s^{\text{atk}}(A) = G_{\text{pre}}(X_v + D \text{diag}(A)), \quad (7)$$

where operator G_{pre} represent all pre-processing. Now, our objective is to optimized A so that the CSA output matches a desired target image x_{tar} .

Differentiable surrogate for optimization. Backpropagating through SBRIM is expensive and slow, so during optimization we replace CSA by a linear Tikhonov inverse operator built from the same $\mathbf{H}$. Starting from the regularized least-squares problem

$$\hat{\mathbf{x}} = \arg \min_{\mathbf{x}} \|\mathbf{y}_s^{\text{atk}} - \mathbf{H}\mathbf{x}\|_2^2 + \lambda_{\text{lin}} \|\mathbf{x}\|_2^2, \quad (8)$$

the normal equations are

$$(\mathbf{H}^H \mathbf{H} + \lambda_{\text{lin}} I) \hat{\mathbf{x}} = \mathbf{H}^H \mathbf{y}_s^{\text{atk}}. \quad (9)$$

Solving for $\hat{\mathbf{x}}$ yields the "linear Tikhonov inverse"

$$W_{\text{CSA}} = (\mathbf{H}^H \mathbf{H} + \lambda_{\text{lin}} I)^{-1} \mathbf{H}^H, \quad (10)$$

so that, for any attacked measurement vector $\mathbf{y}_s^{\text{atk}}(A)$, the surrogate image is

$$\tilde{x}(A) = W_{\text{CSA}} \mathbf{y}_s^{\text{atk}}(A). \quad (11)$$

We then optimize the complex gains A via

$$A^* = \arg \min_A \left(\|\tilde{x}(A) - x_{\text{tar}}\|_2^2 + \lambda_{L2} \|A\|_2^2 \right), \quad (12)$$

using gradient descent, where gradients $\partial \tilde{x}(A) / \partial A$ are obtained through the linear surrogate W_{CSA} .

Final evaluation (true CSA attack). After optimization, we form

$$\mathbf{y}_s^{\text{atk}} = \mathbf{y}_s^{\text{atk}}(A^*) \quad (13)$$

and feed it into the *original* CSA/SBRIM algorithm:

$$\hat{x}_{\text{CSA}}^{\text{atk}} = \text{SBRIM}(\mathbf{y}_s^{\text{atk}}, \mathbf{H}, \lambda_0, p, \eta). \quad (14)$$

Thus, W_{CSA} is used only as a gradient surrogate, whereas the attack performance is always measured on the full CSA reconstructor.

III. BPA SURROGATE IN DI-ATTACK ON LIA

Original LIA (victim mapping). For near-field SAR, let $\mathbf{y}_s \in \mathbb{C}^{M_{\text{meas}}}$ be the vectorized measurements and $\mathbf{x} \in \mathbb{C}^{N_{\text{pix}}}$ the unknown reflectivity image. As in BPA, we build a propagation matrix $\mathbf{H}_{\text{BPA}} \in \mathbb{C}^{M_{\text{meas}} \times N_{\text{pix}}}$ from the sensor and pixel geometry. LIA (Li & Chen) then uses a subsampled version

$$\mathbf{H}_p = \mathbf{H}_{\text{BPA}} \in \mathbb{C}^{K \times N_{\text{pix}}},$$

where $\mathcal{P}$ indexes a subset of K measurements, and applies an iterative matrix-inversion-lemma update to reconstruct

$$\hat{x}_{\text{LIA}} = \text{LIA}(\mathbf{y}_s, \mathbf{H}_p).$$

As in the CSA case, we inject a complex gain vector $A \in \mathbb{C}^{N_p}$ on top of the clean raw data $X_v \in \mathbb{C}^{N_{\text{samp}} \times N_p}$ via a bank of time-domain attack waveforms $D \in \mathbb{C}^{N_{\text{samp}} \times N_p}$:

$$\mathbf{y}_s^{\text{atk}}(A) = G_{\text{pre}}(X_v + D \text{diag}(A)), \quad (15)$$

The victim we are attacking is the original LIA reconstructor

$$\mathbf{y}_s \mapsto \hat{\mathbf{x}}_{\text{LIA}} = \text{LIA}(\mathbf{y}_s, \mathbf{H}_p).$$

BPA surrogate for gradients. Backpropagating through the full LIA iterations is costly, so during the optimization we replace LIA with a simple BPA-style linear inverse built from the same propagation matrix $W_{\text{BPA}} = \mathbf{H}_{\text{BPA}}^H$, i.e., standard back-projection. For any candidate A , the surrogate image is

$$\tilde{\mathbf{x}}(A) = W_{\text{BPA}} \mathbf{y}_s^{\text{atk}}(A), \quad (16)$$

and we solve

$$A^* = \arg \min_A \left(\|\tilde{\mathbf{x}}(A) - \mathbf{x}_{\text{tar}}\|_2^2 + \lambda_{L2} \|A\|_2^2 \right) \quad (17)$$

by gradient descent, using W_{BPA} as a differentiable surrogate for $\text{LIA}(\cdot)$. After convergence, we form the attacked measurements

$$\mathbf{y}_s^{\text{atk}} = \mathbf{y}_s^{\text{atk}}(A^*) \quad (18)$$

and feed them into the original LIA algorithm:

$$\hat{\mathbf{x}}_{\text{LIA}}^{\text{atk}} = \text{LIA}(\mathbf{y}_s^{\text{atk}}, \mathbf{H}_p). \quad (19)$$

Thus, as with CSA, BPA is used *only* as a surrogate for gradients, while the actual victim is the full iterative LIA reconstructor.